

MoCHII User Guide

Abstract

How to build and run MoCHII (MOnte Carlo for H II regions): requirements, the complete input-parameter reference, output-file formats, worked examples keyed to the regression tests, and the runbook for adding an ion. The physics and validation are documented separately in docs/MoCHII_physics.pdf; the atomic-data fitting pipeline in docs/MoCHII_fitting.pdf.

Contents

1. Requirements and build	2
2. Input parameters	2
2.1 Photons and RNG	2
2.2 Grid and dust density	3
2.3 Ionizing/FUV band and the source	5
2.4 Spectrum-file formats	7
2.5 Initial state and iteration	9
2.6 Convergence of the gas iteration	9
2.7 Thermal balance and metals	10
2.8 I-front re-refinement	11
2.9 Dust in the band and dust emission	12
2.10 Peel-off imaging	13
2.11 Output	13
3. Output files	13
3.1 <base>_rates.h5	13
3.2 Line and emissivity outputs	14
3.3 Spectral and image outputs	16
3.4 Reading outputs in Python	16
3.5 Slices and cutouts	17
3.6 Building AMR grids	17
4. Worked examples (the regression tests)	18
5. Adding an ion	19
6. Practical notes	19

1. Requirements and build

- Intel oneAPI MPI Fortran (mpiifort/mpiifx; GNU mpif90 supported by the Makefile), HDF5 (default prefix /data/opt/hdf5_intel; override with `make HDF5_PREFIX=...`), CFITSIO.
- The SEDust dust-emission library, self-contained under SEDust/ in this tree. Build it once with `cd SEDust/sed && ./build_lib.sh` (→ SEDust/sed/lib/libsedust.a and its .mod files, which the Makefile links). The SEDust *data* directory is read at run time and resolved automatically relative to the executable (Section 2.9).
- Python 3 with numpy/scipy (h5py, astropy, matplotlib, PyNeb for the test and pipeline scripts).

Build:

```
cd MoCHII
(cd SEDust/sed && ./build_lib.sh) # on a fresh checkout
make # -> MoCHII.x (always a full clean rebuild)
make DEBUG=1 # bounds/traceback build
```

The SEDust library only needs the one-time `build_lib.sh` step; afterward `make` links the in-tree SEDust library. The Makefile intentionally performs a full rebuild every time (the MoCafe policy): there is no inter-module dependency tracking, and stale .mod files after editing `define.f90` otherwise cause runtime namelist errors.

Run:

```
mpirun -np 8 ./MoCHII.x input.in
```

MoCHII is MPI-only; the octree and all leaf arrays live in MPI-3 shared memory (one copy on each node).

2. Input parameters

All input is one `¶meters` namelist assigning fields of `par`. Parameters not listed keep the defaults shown. The switches compose: `gas_niter = 0` gives a single rates-only pass; `gas_niter > 0` iterates the ionization; `solve_te` adds the thermal balance; `use_metals` the trace-metal registry; `diffuse_field` the explicit diffuse field (forces case A); `refine_front` the I-front re-refinement; `ion_add_dust` the dust coupling.

2.1 Photons and RNG

parameter	default	meaning
no_photons	1e5	source packets in each iteration
no_print	1e7	progress-print stride
iseed	0	RNG seed (0 = from clock/pid)

2.2 Grid and dust density

parameter	default	meaning
grid_type	'car'	'amr' (octree) or 'car' (single-level Cartesian, leaf list in raster order; no tree and no geometry arrays in memory — cell centers and half-widths are computed analytically from the raster index; refine_front unavailable). 'uniform' is accepted as an alias for 'car'
car_walk	'dda'	car-grid traversal: 'dda' (default) = dedicated incremental Amanatides–Woo walks ($t_{\max}/t_{\Delta}$ per axis, comparisons and additions only — the fastest car walk); 'shared' = the octree walks with integer index arithmetic replacing the neighbor table, a verification mode that is bit-identical to 'amr'
amr_file	''	generic AMR leaf file (FITS/HDF5/text): columns x, y, z, level, nH [, metallicity, xHI, ndust]. If empty, a 'car' grid is built from the namelist instead ('amr' requires the file)
nx, ny, nz	1, 1, 11	'car'-from-namelist cell counts (cubic cells required: $2x_{\max}/n_x = 2y_{\max}/n_y = 2z_{\max}/n_z$)
xmax, ymax, zmax	1, 1, 1	'car'-from-namelist box half-extents [distance_unit]; box is $[-x_{\max}, x_{\max}]$ etc.
nH_const	-1	if ≥ 0 , override every leaf's n_H with this uniform value [cm^{-3}] (both 'car' and 'amr'); < 0 keeps the file/column density
density_file	''	read the leaf n_H from a uniform 3D density cube onto a 'car' grid (FITS 3D image, or the MoCHII HDF5 image format; NAXIS1/2/3 = $n_x/n_y/n_z$, values n_H [cm^{-3}]). $n_x/n_y/n_z$ come from the file, the box is xmax/ymax/zmax; an alternative to amr_file. reduce_factor > 1 down-samples; a rmax/rmin cut can mask the cube to a sphere
rmax, rmin	-999, 0	spherical density cut on leaf centers (both grids): $n_H = 0$ for $r > r_{\max}$ ($r_{\max} > 0$) and for $r < r_{\min}$ ($r_{\min} > 0$; a shell); radii in distance_unit
distance_unit	'kpc'	'kpc'/'pc'/'au'/'' (code units)

parameter	default	meaning
dust_model	'global_dgr'	leaf grey opacity: 'none', 'global_dgr' ($n_{\text{H}}c_{\text{ext}}\cdot\text{DGR}$), 'from_file' (ndust column), 'laursen09' (static x_{HI} column), 'laursen09_live' (computed x_{HI} , refreshed each iteration)
cext_dust	1.6059e-21	reference extinction per H [cm^2] at λ_{ref}
DGR, Z_global, Z_ref	1e-2, 0.0134, 0.0134	dust-model scalings
f_ion_dust	0.01	laursen09 survival of dust in ionized gas
f_pah, f_ion_pah	0, 0	PAH share of the reference extinction and its own ionized-gas survival (laursen09_live)

2.3 Ionizing/FUV band and the source

parameter	default	meaning
use_ion_band	F	must be T (the MoCHII transport)
nnu_ion	16	ionizing-band bins; gates use 32. With ion_align_edges on (the default) the bins are log-uniform within each threshold-bounded segment; off, they are a single log grid
eion_min, eion_max	13.598, 100	ionizing band edges [eV]
ion_align_edges	T	pin the ionizing-band bin edges onto the ionization thresholds (H I 13.6, He I 24.6, He II 54.4 eV plus the active metal edges) so no bin straddles one; segments between thresholds are log-uniform, and if the thresholds outnumber the bins the lowest-priority metal edges are dropped (H/He always kept), so it is safe at any nnu_ion. Removes the $\sim 18\%$ He-ionizing-photon excess of a coarse log grid crossing the 24.6 eV edge. Set F for the legacy single log grid
add_fuv	F	FUV extension: extra log bins on [efuv_min, eion_min] below the unchanged ionizing bins; transports FUV grain heating and FUV photoionization of low-threshold metals (Mg I, C I, Si, Fe I); luminosity stays the IONIZING luminosity (Q_H preserved, the FUV part rides on top)
efuv_min	6.0	FUV lower edge [eV]
nnu_fuv	8	FUV bins (16 recommended with metals)
tstar	-999	Planck source temperature [K]
ion_spectrum	"	global spectrum file (column 1 plus one value column); its column units are set by spectrum_type (§2.4)
spectrum_type	'shape'	column units of every spectrum-file slot (ion_spectrum, src_spectrum_file, ext_spectrum): 'shape' (arbitrary, normalized to the scale — legacy) or a physical type 'per_ev'/'per_hz'/'per_ang'/'per_um' (absolute); see §2.4
luminosity	unset	ionizing BAND luminosity [erg/s] of the (single) source; for a 'shape'/Planck source unset maps to 1.0 (legacy), for a physical-type file unset means DERIVE it from the file integral; with $n_{\text{source}} \geq 2$ it becomes the sum of src_lum (or the equal-split total when those are unset)
xs_point, ys_point, zs_point	0	single-source position (code units, recentered box; $n_{\text{source}}=1$)

Multiple internal point sources (the scalars above are the $n_{\text{source}}=1$ case):

parameter	default	meaning
nsource	1	number of internal point sources (0–16); 1 keeps the legacy scalars above, ≥ 2 uses the <code>src_*</code> arrays
src_x, src_y, src_z	0	positions of sources 1..nsource (code units)
src_lum	unset	ionizing luminosities [erg/s] of the sources; set all or none (none = equal split of luminosity); a partial set aborts
src_tstar	–999	Planck temperature [K] for each source (multi-source runs only; a single source uses <code>tstar</code>)
src_spectrum_file	”	one multi-column spectrum file for all sources: column 1 then one value column for each source (units set by <code>spectrum_type</code> , §2.4); each source is independently normalized so its ionizing segment carries <code>src_lum(i)</code> (a physical type with <code>src_lum</code> unset is derived from that column); overrides <code>src_tstar</code> ; too few columns abort, extra columns are ignored with a note

External isotropic field (composes with the internal sources; ON when `ext_intensity` > 0, an ISRF preset is named, or a physical-type `ext_spectrum` file is given):

parameter	default	meaning
ext_intensity	-1	band-integrated mean intensity J [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$] of the external field; the field is ON when > 0 and adds to the internal sources; entering ionizing power $L = \pi J A_{\text{surf}}$. With a physical-type ext_spectrum file or an ISRF preset it is optional: unset = the absolute J is DERIVED from the file/preset, set = the field is RESCALED to this band J (a note is printed)
ext_geometry	'rec'	illuminating surface: 'rec' (box faces, face-area-weighted — correct for a non-cubic box) or 'sph' (bounding sphere of radius rmax about the box center; needs $r_{\text{max}} > 0$)
ext_tstar	-999	external-field Planck temperature [K]
ext_spectrum	"	external-field spectrum: a 2-column file (units set by spectrum_type, §2.4; column 2 is an intensity density J) OR a reserved ISRF preset name 'draine'/'habing'/'mathis' (FUV-only, absolute, needs add_fuv); priority $\text{ext_spectrum} > \text{ext_tstar} >$ the global spectrum
ext_scale	1.0	dimensionless multiplier on an ISRF preset (ignored when $\text{ext_intensity} > 0$, which rescales the preset to a target band J)
source_geometry	'point'	legacy external-only shorthand: 'external', 'external_rec', or 'external_sph' forces $n_{\text{source}}=0$ with the external field on (needs $\text{ext_intensity} > 0$); 'point' keeps the composable form

2.4 Spectrum-file formats

A single parameter, spectrum_type (default 'shape'), fixes the column units of *every* spectrum-file slot (ion_spectrum, src_spectrum_file, ext_spectrum). The type sets the units; the slot sets the meaning: an internal-source file gives a luminosity density L_X , the external-field file gives an intensity density J_X . Column 1 and the value columns are:

spectrum_type	column 1	internal (cols 2+)	external (col 2)
'shape'	E [eV] $\uparrow$	arbitrary per eV (shape of L_E)	arbitrary per eV
'per_ev'	E [eV]	L_E [$\text{erg s}^{-1} \text{eV}^{-1}$]	J_E
'per_hz'	ν [Hz]	L_ν [$\text{erg s}^{-1} \text{Hz}^{-1}$]	J_ν
'per_ang'	λ [Å]	L_λ [$\text{erg s}^{-1} \text{Å}^{-1}$]	J_λ
'per_um'	λ [μm]	L_λ [$\text{erg s}^{-1} \mu\text{m}^{-1}$]	J_λ

The external intensity density J_X carries the same spectral unit as the internal L_X times an extra $\text{cm}^{-2} \text{sr}^{-1}$ (a mean inten-

sity). All files are plain text with '#' comment lines; values outside the tabulated range are zero. `src_spectrum_file` is one multi-column file: column 1 shared by all sources, then one value column for each source (extra columns beyond `nsource` are ignored with a note, too few abort).

The abscissa is a sample grid, not bin centers. Column 1 lists the points at which the spectral density is *sampled*: the file describes a continuous function, defined between the points by linear interpolation, and no bin widths are attached to the rows. MoCHII integrates each of its own band bins on a 32-point sub-grid of the bin, interpolating the table at every sub-point (zero outside the tabulated range), so the file grid and the band grid are independent. The spacing is arbitrary — nonuniform grids are fine; place points densely where the spectrum varies rapidly, since only linear variation is captured between adjacent points. Abscissa values must be distinct (no duplicates); a 'shape' file must be ascending in E , while the physical types accept any row order (the table is sorted internally after the unit conversion, so a wavelength table may be given wavelength-ascending).

Absolute vs. shape normalization. A physical type ('per_ev'/'per_hz'/'per_ang'/'per_um') is *absolute*: the bin luminosities follow the tabulated integral directly. If the matching scale (`luminosity / src_lum(i) / ext_intensity`) is unset it is DERIVED from the file — each source from its column's ionizing-segment integral, the external field from its band-integrated J — and reported in the log. If a scale is set, the spectrum is RESCALED to it (a note is printed) and the file then acts only as a shape. With `add_fuv` an absolute spectrum fills the FUV bins from the same table, with no separate normalization. 'shape' keeps the legacy behavior: the shape is normalized to the scale in every case. A spectrum with no luminosity inside the band aborts, as does rescaling a file whose ionizing-segment integral is zero.

'shape' is per eV, not variable-agnostic. 'shape' reads its value columns as the shape of the per-eV density L_E on the E [eV] grid of column 1. A table of F_λ (or F_ν) values re-gridded onto E under 'shape' therefore omits the Jacobian and is wrong by a factor E^{-2} (the $d\lambda/dE$ conversion). To use a wavelength- or frequency-tabulated shape, give it with its matching physical type and set the scale explicitly — the type conversion then applies the Jacobian and the scale RESCALES the file to your target, so the file acts as a shape with the correct units. For example, a wavelength shape normalized to 10^{49} ionizing photons' worth of luminosity:

```
par%spectrum_type = 'per_ang'
par%ion_spectrum   = 'wave_shape.txt'
par%luminosity     = 3.0e38
```

A small 'per_ang' source file for two point sources (λ ascending; the second source is FUV-faint):

#	lambda[A]	L_lam,1	L_lam,2
	300.0	7.80e34	3.4e33
	500.0	6.05e34	1.2e33
	912.0	4.10e34	0.0

2.5 Initial state and iteration

parameter	default	meaning
xHI_init	1.0	initial $n_{\text{HI}}/n_{\text{H}}$ (an xHI file column wins); an ionized start (1e-3) converges much faster than neutral
xHeI_init, xHeII_init	1, 0	initial He fractions
He_abund	0.1	$n_{\text{He}}/n_{\text{H}}$
gas_niter	0	iterations (0 = a single rates-only pass)
gas_tol	1e-3	convergence tolerance on x_{HII} (measure set by conv_crit)
conv_crit	'cell'	convergence measure: 'cell' (cell maxima; stalls at the front-cell noise floor) or 'vol' (volume-integrated L_1 ; noise-robust, recommended). See §2.6
ion_relax	1.0	under-relaxation on x (< 1 damps front oscillation)
case_ab	'B'	case A/B recombination ('A' forced by the diffuse field); also selects the SH95 H I/He II line tables (case-A/B)
require_convergence	F	if the gas iteration has not converged at gas_niter, print a warning; when T, abort before writing output. The convergence status is recorded in the _rates header either way (CONVERGD, NITERDON, FINALDX, FINALDTE)

2.6 Convergence of the gas iteration

The gas iteration (gas_niter sweeps of transport $\rightarrow$ rate integrals $\rightarrow$ leaf solve $\rightarrow$ opacity refill) stops when the state stops changing between sweeps. Each sweep compares a *change measure* against a tolerance:

- ionization only (te_fixed, no thermal solve): converged when the x_{HII} change $<$ gas_tol;
- coupled (solve_te): converged when the x_{HII} change $<$ gas_tol *and* the T_e change $<$ gas_tol_te.

Two measures (conv_crit). Both are computed every sweep; conv_crit selects which one gates the loop.

'cell' the cell maxima $\max_{\text{leaf}} |\Delta x_{\text{HII}}|$ and $\max_{\text{leaf}} |\Delta T_e|/T_e$. Simple, but the ionization-front cells jitter with Monte Carlo noise and never settle, so $\max |\Delta x_{\text{HII}}|$ *stalls* at the front-cell noise floor ($\sim 2\text{--}4 \times 10^{-2}$ at 4×10^7 packets) and a smaller gas_tol is unreachable. This is the historical default (kept so the recorded gates reproduce).

'vol' the volume-integrated L_1 changes

$$\frac{\sum_{\text{leaf}} V |\Delta x_{\text{HII}}|}{\sum_{\text{leaf}} V x_{\text{HII}}} < \text{gas_tol}, \quad \frac{\sum_{\text{leaf}} V n_e |\Delta T_e|}{\sum_{\text{leaf}} V n_e T_e} < \text{gas_tol_te}.$$

Front jitter occupies little volume (a few thousand front cells against $\sim 10^5$ ionized ones), so it enters the norm suppressed by that ratio; the n_e weight drops the vacuum and te_min-pinned no-heating cells that otherwise dominate $\max |\Delta T_e|$ in FUV/PDR runs. These measures decay smoothly — the recommended production setting.

The Monte Carlo floor. Even the volume measures do not reach zero: they floor at a Monte Carlo level set by the packet count, scaling as $1/\sqrt{N_{\text{packets}}}$ ($\sim 3 \times 10^{-3}$ in x and $\sim 7 \times 10^{-3}$ in T_e at 8×10^6 packets). Reaching the floor is

convergence — iterating further leaves the solution unchanged. So **set `gas_tol/gas_tol_te` just above the floor for the packet count in use**: a tolerance below the floor can never be met and the loop runs to `gas_niter` and warns. Rules of thumb (`conv_crit='vol'`): $\sim 5 \times 10^{-3}/10^{-2}$ at $\sim 10^7$ packets, loosened to $\sim 10^{-2}/2 \times 10^{-2}$ at $\sim 10^6$ (the reduced-count examples/ use the looser values for this reason).

When it does not converge. Hitting `gas_niter` without meeting the test is not fatal: the state is still written and a warning is printed on rank 0. Set `require_convergence = T` to abort before the output instead. Either way the `_rates` header records the outcome in `CONVERGD` (0/1), `NITERDON` (sweeps run), `FINALDX`, and `FINALDTE`. The derivation and the gate measurements are in `docs/MoCHII_physics.pdf`.

2.7 Thermal balance and metals

parameter	default	meaning
<code>solve_te</code>	F	bisection on T_e coupled with the ionization
<code>te_fixed</code>	1e4	fixed T_e [K] when <code>solve_te</code> is off
<code>te_min, te_max</code>	1e3, 5e4	bisection bracket [K]
<code>gas_tol_te</code>	1e-3	convergence on $\max \Delta T_e /T_e$
<code>atomic_dir</code>	'data/atomic'	coefficient files (relative to the run directory)
<code>use_metals</code>	F	registry cascade + metal line cooling
<code>ci_model</code>	'voronov'	collisional ionization: 'voronov' (default) or 'dere_hybrid' (Voronov $\times$ the Dere 2007/Voronov ratio; matters only in shielded low-ionization zones)
<code>abund_C/N/O/Ne/S</code>	HII20 values	$n(X)/n(H)$; an element is active when > 0
<code>abund_Ar/Mg/Fe</code>	0	further registry elements (gas-phase values; Mg/Fe are heavily depleted)
<code>abund_Si/Cl/Ca</code>	0	five-stage registry elements (gas-phase; Si and especially Ca heavily depleted, Cl nearly undepleted — Orion-like $3e-6 / 3e-7 / 2e-8$); adds [Si II] 34.8 μm , Si III/IV, [Cl II/III/IV] (5518/5538 density pair), Ca II, [Ca V]
<code>ion_metal_abs</code>	T	metal photoionization absorption in the band opacity (active with <code>use_metals</code>); the only gas opacity in the FUV bins

parameter	default	meaning
emis_output	F	write <code>_emis</code> (HDF5/FITS): leaf-by-leaf line emissivities + the state needed for maps (Section 3.)
h_coll_effects	F	add the collisional H I Balmer component (excitation of neutral H) as separate hc rows/blocks; matters at fronts and hot cells
fe_levels_full	F	load the expanded Fe II/III models (<code>nlevel_fe_{2,3}_full.txt</code> , 16/34 levels) instead of the compact 13/14 for iron-line studies
diffuse_field	F	explicit ground-recombination packets
hei_diffuse	F	fourth diffuse channel (needs <code>diffuse_field</code>): He I excited-level recombination photons (19.82 eV, 584 Å, two-photon) — correct energy bookkeeping; moves Te by $\sim 1\%$ at HII40, He split unchanged
gaunt_vh14	F	free-free Gaunt factors from the van Hoof et al. (2014) table (<code>data/gauntff_vh14.dat</code>) instead of Hummer (1988); the two agree to $\lesssim 0.5\%$ in T_e
metal_ne	F	PDR: electrons from the metal cascade join the n_e closure (the only electrons beyond the I-front)
metal_heat	F	PDR: metal photoheating in the thermal balance
grain_pe	F	PDR thermal package: grain photoelectric heating + recombination cooling (Bakes & Tielens 1994) from the local FUV field (needs <code>add_fuv</code>), plus H-impact [C II] 158 μm / [O I] 63 μm cooling; scaled by <code>pe_scale</code> and the leaf dust amount
pe_scale	1.0	photoelectric-efficiency scale factor

2.8 I-front re-refinement

parameter	default	meaning
refine_front	F	rebuild the octree on the I-front once
refine_iter	20	at which iteration
refine_lbase, refine_lmax	5, 7	base/front levels
refine_eps	1e-2	front flag: $\epsilon < x_{\text{HI}} < 1-\epsilon$
refine_dx	0.2	or a face-neighbor x_{HI} jump above this

2.9 Dust in the band and dust emission

parameter	default	meaning
ion_add_dust	F	dust absorption in the band (dusty Strömgren)
ion_dust_kext	"	kext table with EUV coverage (e.g. data/kext_albedo_WD_MW_3.1_60_D03.all_2009); required
ion_dust_scatter	F	sample dust scattering (HG, albedo weights)
dust_sed	F	SEDust stochastic dust SED at output time
dust_model_sed	'astrodust'	'astrodust'/'dl07'/'zubko'
sed_workdir	" (auto)	SEDust sed/ tree whose dielectric tables are read relative to it; blank auto-resolves to <executable dir>/SEDust/sed (the in-tree copy), so it normally needs no setting
sed_qtable, sed_sizedist	in-tree defaults	optics/size-distribution tables (relative to sed_workdir)
sed_NT, sed_Tlo, sed_Thi	200, 2.7, 5e3	SEDust temperature grid
dust_emis_bands	unset	up to 8 wavelengths [μm] (e.g. 8, 24, 70, 160): SEDust band emissivities of each leaf $\rightarrow$ _dustemis (blocks emis_dust/wl_dust [$\text{erg s}^{-1} \text{cm}^{-3} \mu\text{m}^{-1}$]); the Python map maker projects them into dust-band images (the IR is optically thin)
sed_pah_live	F	x_{HII} -dependent PAH survival in the emission: the leaf spectrum is assembled from the model channels with the PAH channel weighted by $(x_{\text{HI}} + f_{\text{ion,pah}} x_{\text{HII}})$ (needs a model with a 'PAH' channel, i.e. astrodust); turns the 8- μm map into the classic PAH ring at the ionization front

2.10 Peel-off imaging

parameter	default	meaning
ion_peel	F	after convergence, one extra transport pass peels direct (stellar + diffuse) and dust-scattered contributions toward the observers $\rightarrow$ <code>_image</code> (blocks <code>direct_ion/scatt_ion</code> , plus <code>_fuv</code> with <code>add_fuv</code>) [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$]
peel_bins	F	bin-resolved cubes <code>direct_cube/scatt_cube</code> (n_x, n_y, n_ν) in addition to the band-integrated channel images (hardness maps, synthetic EUV/FUV photometry); the third (bin) axis is labelled by the <code>E_bin/dE_bin</code> [eV] and <code>wl_bin</code> [$\text{\AA}$] arrays written once into the same file
save_direct0	F	also write the unattenuated direct image (<code>direct0_ion/_fuv</code> , and <code>direct0_cube</code> with <code>peel_bins</code>): the direct source with the path extinction $e^{-\tau}$ omitted, so <code>direct0/direct</code> = $e^{+\tau}$ is the line-of-sight optical depth toward the observer
obsx, obsy, obsz	unset	observer positions (or directions) in code units; or use <code>alpha/beta/gamma</code> angle lists [deg]; default: one observer on +z far away
distance	auto	observer distance (code units)
nxim, nyim	129, 129	image pixels
dxim, dyim	auto	pixel scale [deg]; auto-sized to cover the box

Peel-off also works with the external field: an external packet's direct peel uses the Lambert weight $\max(\hat{k}_{\text{obs}} \cdot \hat{n}_{\text{in}}, 0)/\pi$ about its entry-surface inward normal, so only the far-side surface contributes and the image is the background field seen through the medium (the cloud in silhouette); with `save_direct0`, `direct0/direct` = $e^{+\tau}$ is again the line-of-sight optical-depth map.

2.11 Output

`out_file` (base name; extension follows `file_format` = 'hdf5' or 'fits').

3. Output files

3.1 <base>_rates.h5

One dataset for each extension, leaf-ordered:

extension	content
Gamma_HI/HeI/HeII	photoionization rates [s^{-1}]
Heat_HI/HeI/HeII	photoheating per particle [erg/s]
Heat_dust	grain heating [$\text{erg s}^{-1} \text{cm}^{-3}$] (when ion_add_dust)
T_dust	equilibrium mixture dust temperature [K]
x_HI, x_HeI, x_HeII, n_e, T_e	converged state
x_<el>_stages	metal ion fractions (stage, leaf)
J_nu	band mean intensity (bin, leaf) [$\text{erg/s/cm}^2/\text{Hz/sr}$]
E_bin, dE_bin	band grid [eV]
LeafXYZ, LeafSize	leaf centers and full widths (code units; for slices/cutouts)

The file header also carries the convergence record of the gas iteration: CONVERGD (did it converge), NITERDON (iterations run), FINALDX (final max $|\Delta x_{\text{HII}}|$), and FINALDTE (final max $|\Delta T_e|/T_e$).

3.2 Line and emissivity outputs

<base>_lines.txt is a text table of integrated line luminosities on the converged state, one row for each line: element, ionization stage, wavelength [$\text{\AA}$], L [erg s^{-1}], and $L/L(\text{H}\beta)$. With emis_output the same line set is also written leaf-by-leaf to <base>_emis (HDF5, or FITS with file_format='fits') as emissivity blocks in $\text{erg s}^{-1} \text{cm}^{-3}$, where $\sum \text{emis} \cdot V = L_{\text{line}}$ and the block row order matches the text table. Every line comes from one of two sources: recombination-line tables (H and He, Table 1) and n -level statistical-equilibrium solves (the metals plus the optional collisional H I component, Table 2).

Table 1: Recombination-line blocks in <base>_emis; wavelengths in $\text{\AA}$ unless μm noted.

block	source	lines	weighting, present	when
emis_h	Storey & Hummer (1995) H I, case set by case_ab	H α 6565, H β 4863, H γ 4342, H δ 4103, Pa α 1.876 μm , Pa β 1.282 μm , Br γ 2.166 μm	always; $n_e n_{\text{HII}}$	
emis_heii	SH95 He II (case per case_ab)	1640, 3204, 4543, 4686, 5413, 10126	only where an He III zone exists; $n_e n_{\text{HeIII}}$	
emis_hei	Porter et al. (2012/2013) He I, case B	3889, 4026, 4471, 5876, 6678, 7065, 7281, 10830	where He II is present; $n_e n_{\text{HeII}}$	
emis_hc	25-level H I collisional solve	H α /H β /H γ collisional component (separate hc rows)	only h_coll_effects; n_{HI}	with

The He I Porter table is case B only; under case A a note is printed and the H I/He II tables switch to case A while He I stays case B.

The metal line blocks emis_<el>_<stage> follow the n -level solve. One block is written for every active registry ion (element abundance > 0) that has an nlevel_<el>_<stage> data file. Elements C, N, O, Ne, S are active by default;

Ar, Mg, Fe, Si, Cl, Ca are activated by setting their `par%abund_<El>`. Only the principal lines are listed below — the full transition list of each block is in `<base>_lines.txt`.

Table 2: Metal collisional-line blocks (`emis_<el>_<stage>`); wavelengths in Å unless μm noted.

ion (block)	principal lines
c_2 [C II]	2325 (UV multiplet), 158 μm
c_3 [C III]	977, 1907, 1909, 178 μm
n_2 [N II]	3064/3071, 5756, 6550, 6585, 122 μm , 205 μm
n_3 [N III]	1750 (UV multiplet), 57 μm
o_1 [O I]	5579, 6302, 6366, 63 μm , 145 μm
o_2 [O II]	2471, 3727, 3730, 7321/7331
o_3 [O III]	1661/1666, 2322/2332, 4364, 4960, 5008, 52 μm , 88 μm
ne_2 [Ne II]	12.8 μm
ne_3 [Ne III]	3343, 3870, 3969, 15.55 μm , 36 μm
s_2 [S II]	4070/4078, 6718, 6733, 1.03 μm , 315 μm
s_3 [S III]	3722, 6314, 8831, 9070, 9532, 12 μm , 18.7 μm , 33.5 μm
<i>Optional — activate with <code>par%abund_<El></code></i>	
ar_3 [Ar III]	3006/3110, 5193, 7138, 7753, 8.99 μm , 21.8 μm
ar_4 [Ar IV]	2854/2869, 4713, 4741, 7170 (multiplet), 56 μm , 78 μm
mg_2 [Mg II]	2796, 2804
fe_2 [Fe II]	1.26/1.32 μm , 5.34 μm , 17.9 μm , 26 μm (40 lines total)
fe_3 [Fe III]	4659, 4703, 4881, 5011, 22.9 μm (38 lines; 16/34-level with <code>fe_levels_full</code>)
si_2 [Si II]	2335 (multiplet), 34.8 μm
si_3 [Si III]	1206, 1883, 1892, 38 μm
si_4 [Si IV]	1394, 1403
cl_2 [Cl II]	3587/3679, 6164, 8581, 9126, 14.4 μm , 33 μm
cl_3 [Cl III]	3344/3354, 5519, 5539, 8480 (multiplet)
cl_4 [Cl IV]	3120/3205, 5325, 7263, 7533, 11.8 μm , 20.3 μm
ca_2 [Ca II]	3935, 3970, 7293/7326, 8500/8544/8665
ca_5 [Ca V]	3999, 5311, 6088, 3.05 μm , 4.16 μm , 11.5 μm

Some registry ions are tracked in the ionization balance but carry no line block because CHIANTI v11 has no level/collision data for them: Ar II, Cl V, Ca III, Ca IV.

State blocks. Beyond the emissivities, `<base>_emis` carries the state a map needs without the rates file: `LeafXYZ` (leaf centers) with the `DIST_CM` keyword, `LeafSize` (leaf edge length; $V = (\text{LeafSize} \cdot \text{DIST_CM})^3$), `n_H`, `n_e`, `T_e`, `x_HeI`, `x_HeII`, and the `x_<el>_stages` metal-fraction blocks (same layout as the rates file). The Python `EmisData` reader (`tools/python/mochii_output.py`) turns these blocks into line maps without the rates file.

3.3 Spectral and image outputs

file	content
_nebcont.txt	nebular continuum L_ν (fb+ff tables, Milne, two-photon)
_dustir.txt	equilibrium-mixture dust IR spectrum (1–1000 μm) with the energy-closure header
_dustsed.txt	SEDust stochastic dust SED with PAH bands (when dust_sed)
_dustemis.h5	(with dust_emis_bands) SEDust band emissivities of each leaf, same block layout as _emis — readable by EmisData for dust-band maps
_image.h5	(with ion_peel) peel-off images for each observer: direct_ion/scatt_ion (+_fuv) [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$], suffix _obs<k> when several observers; with peel_bins, direct_cube/scatt_cube (n_x, n_y, n_ν) and the E_bin/dE_bin/wl_bin bin-grid arrays; with save_direct0, the unattenuated direct0_ion/_fuv (or direct0_cube)

3.4 Reading outputs in Python

tools/python/mochii_output.py is an importable module (works directly in a notebook) that reads every section file (HDF5 or FITS; read_sections), the line table (read_lines_table), and the text spectra (read_spectrum), and builds 2D maps from the emissivity file:

```
import sys; sys.path.insert(0, ".../MoCHII/tools/python")
import matplotlib.pyplot as plt
from mochii_output import EmisData

d = EmisData("run_emis.h5")
d.info()                # blocks, fields, line count
d.line_list()           # every stored line (block, wl)

# ready-made figures (notebook): returns (fig, ax)
fig, ax = d.plot_line("o_3", 5007.)          # log S map
fig, ax = d.plot_line("h", 6563., unit="intensity",
                    photons=True) # photons/s/cm^2/sr
fig, ax = d.plot_field("T_e", weight="EM")
fig, axs = plt.subplots(1, 2); d.plot_line("s_2", 6717., ax=axs[0]) \
    ; d.plot_line("s_2", 6731., ax=axs[1]) # doublet side by side

# raw arrays, when the figure is yours to build
img, ext = d.line_map("o_3", 5007., axis="z", npix=512)
img, ext = d.line_map("h", 6563., unit="intensity", photons=True)
img, ext = d.field_map("T_e", weight="EM") # LOS-weighted average
```



```
Q = d.line_luminosity("h", 4861., photons=True) # photon rate [1/s]
```

Line maps are flux-conserving: each leaf's luminosity is deposited with exact area overlap onto the pixel grid (AMR leaf sizes handled), so $\sum SA_{\text{pix}} = L_{\text{line}}$ to machine precision. `unit='flux'` (default) gives surface brightness [$\text{erg s}^{-1} \text{cm}^{-2}$]; `unit='intensity'` gives specific intensity [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$] = $S/4\pi$ (isotropic emission, optically thin); `photons=True` converts either to photon units (divides by hc/λ of the line). A `slab=(lo,hi)` cut restricts the line of sight. Field maps average `T_e`, `n_e`, `x_HI`, ... along the line of sight with weight `'EM'` ($n_e^2 V$), `'ne'`, `'nH'`, or `'V'`. The same module doubles as a command-line tool for quick looks:

```
python3 mochii_output.py run_emis.h5 # list lines
python3 mochii_output.py run_emis.h5 --line o_3 5007 \
    --npix 512 --log --out o3.png
python3 mochii_output.py run_emis.h5 --line h 6563 \
    --unit intensity --photons # photons/s/cm^2/sr
python3 mochii_output.py run_emis.h5 --field T_e --weight EM
```

3.5 Slices and cutouts

A *slice* is the actual leaf value on a plane (the cell that contains each pixel), distinct from the line-of-sight *projection* above — a slice resolves the ionization front sharply where a projection blurs it. `tools/python/leaf_field.py` holds the backend (`leaf_slice`, the nearest-leaf fallback `leaf_slice_nn`, and `plot_slice`); `mochii_output.py` exposes it on the output files:

```
from mochii_output import EmisData, RatesData
r = RatesData("run_rates.h5") # leaf state
fig, ax = r.plot_field_slice("T_e", axis="z", coord=0.0)
fig, ax = r.plot_field_slice("x_HI", axis="z", coord=0.0, log=True)
img, ext = r.slice_field("n_e", axis="x", coord=0.0,
    bounds=(-2,2,-2,2)) # cutout window

d = EmisData("run_emis.h5")
fig, ax = d.plot_line_slice("o_3", 5007., axis="z") # emissivity slice
```

Exact slices need the leaf full widths: the `_emis.h5` file always carries `LeafSize`, and `_rates.h5` does too; a rates file written before `LeafSize` existed falls back to a nearest-leaf slice (a one-time note is printed). The command line takes `--slice FIELD --axis z --coord 0` on either file.

3.6 Building AMR grids

`tools/python/AMR_grid/` builds the generic AMR octree file the code reads (`grid_type='amr'`): `AMR_grid.py` (the `AMRGrid` class — uniform, density-gradient, or resolution refinement; `set_density/set_neutral_fraction`; write to

HDF5/FITS/.fits.gz/text with columns `x,y,z,level,nH[,metallicity,xHI,ndust]`) and `make_amr_sphere.py` (a sphere/shell CLI). AMRGrid carries the same `slice/plot_slice/cutout` so a grid can be inspected before a run.

4. Worked examples (the regression tests)

Each `tests/` directory is self-contained: grid builder or a symlinked grid, the input file, and a `check_*.py` gate script.

directory	what it demonstrates / gate
<code>g0_gamma</code>	rate integrals vs. analytic attenuation (bias +0.004%)
<code>g1_stromgren</code>	Strömgren sphere, case B fixed T_e ; AMR $\equiv$ uniform
<code>g2a_thermal</code>	thermal H/He nebula vs. the 1D exact reference
<code>g2_hii</code>	MOCASSIN Lexington HII20/HII40 benchmarks + line fluxes
<code>g4_refine</code>	I-front re-refinement vs. native level 7
<code>g4_tng</code>	Illustris-TNG post-processing demo (shadows, maps)
<code>d_dusty</code>	dusty Strömgren; scattering bracket; T_d /IR
<code>peel</code>	peel-off images vs. the optically thin analytic flux (+0.017%); bin cubes
<code>uni_dda</code>	uniform grid: shared walk $\equiv$ octree (bit-identical); DDA walk to rounding
<code>pdr</code>	FUV/PDR switches: carbon-electron shell, photoelectric T_e
<code>ext_field</code>	isotropic external field: interior $J = J_{\text{ext}}$ to 0.2% (rec and sph)
<code>multi_src</code>	multiple point sources: superposition, spectrum resolution, one-file spectra
<code>peel_ext</code>	external-field peel: silhouette flux vs. JA_{proj}/d^2 , chord τ spectrum

A minimal thermal + metals + dust input (from `d_dusty`):

¶meters

```

par%no_photons = 4.0e7          par%iseed = 1234
par%grid_type = 'amr'          par%amr_file = 'amr_L6.fits'
par%distance_unit = 'pc'       par%dust_model = 'global_dgr'
par%use_ion_band = .true.      par%nnu_ion = 32
par%eion_min = 13.598          par%eion_max = 100.0
par%tstar = 4.0e4              par%luminosity = 3.178e38
par%xHI_init = 1.0e-3          par%He_abund = 0.1
par%gas_niter = 60             par%gas_tol = 2.0e-3
par%solve_te = .true.          par%atomic_dir = '../data/atomic'
par%use_metals = .true.
par%ion_add_dust = .true.
par%ion_dust_kext = '../data/kext_albedo_WD_MW_3.1_60_D03.all_2009'
par%cext_dust = 4.868e-22      par%DGR = 1.0
par%out_file = 'run.h5'        par%file_format = 'hdf5'

```

/

5. Adding an ion

Adding an ion touches only data:

1. `tools/fitting/fit_cooling.py`: add the ion with a T_i ladder guessed from its `.elvlc` level energies; run; check the reported max error and the overlay figure.
2. `tools/fitting/fit_nlevel.py`: add the ion with the level count needed for its diagnostics; run.
3. `tools/fitting/make_element_data.py`: add the element to `ELEMENTS` (Z , stage count) and its thresholds; charge exchange entries where sources exist; run. The CHIANTI `.rrparams/.drparams` types 1–3 are handled (RR/RR2/DR/DR2 lines).
4. Fortran: a `par%abund_<El>` field and one entry in the `species_setup` name/abundance lists (only for a NEW element; new stages of a known element need nothing).
5. Verify: extend `verify_nlevel_pyneb.py` with a diagnostic ratio where PyNeb has data; run a smoke test and check the new lines in `_lines.txt`.

Data caveats met so far: Ar II has no CHIANTI v11 level/collision data (stage tracked without lines); Fe III collision strengths differ from PyNeb’s BB14 by up to $\sim 16\%$ in 4658/4702 (known spread for iron); charge-exchange stage labels differ between the Huang (2023) and MOCASSIN transcriptions for Mg/Fe (the Huang labels are adopted; see `make_element_data.py`).

6. Practical notes

- **Bins**: 32 bins give $\sim 2\%$ rate-integral discretization; use 64 for sub-percent work. The `add_fuv` extension carries its own bins below `eion_min`, so the ionizing resolution is unaffected.
- **FUV and the PDR**: without `add_fuv` there is no radiation beyond the I-front and the metals there recombine toward neutral; with it, FUV penetrates the neutral gas (dust extinction only) and produces the PDR-side C II and Mg II. Metal photoheating and grain photoelectric heating are not in the thermal balance, so PDR-zone temperatures are indicative only.
- **Convergence**: use `conv_crit = 'vol'` for production — the cell-max criterion stalls at the front-cell noise floor (and, with `add_fuv`, at no-heating cells where T_e pins to `te_min`). Both measures are printed each iteration. The explicit diffuse field smooths the front and converges far better.
- **Starts**: prefer an ionized initial state; a neutral start advances the front $\sim$ one cell per iteration.
- **next tables**: must cover the band (the astro dust table stops at 912 Å; use the D03 table for EUV work). Tables are sorted on read (the D03 file has an out-of-order seam).
- **Re-refinement**: one event per run; the old shared-memory windows are reclaimed only at exit.

- **SEDust:** `sed_workdir` must point at a SEDust `sed/` tree; the exact stochastic solve costs $\sim 0.05\text{--}0.1$ s for each gas leaf (distributed over ranks).